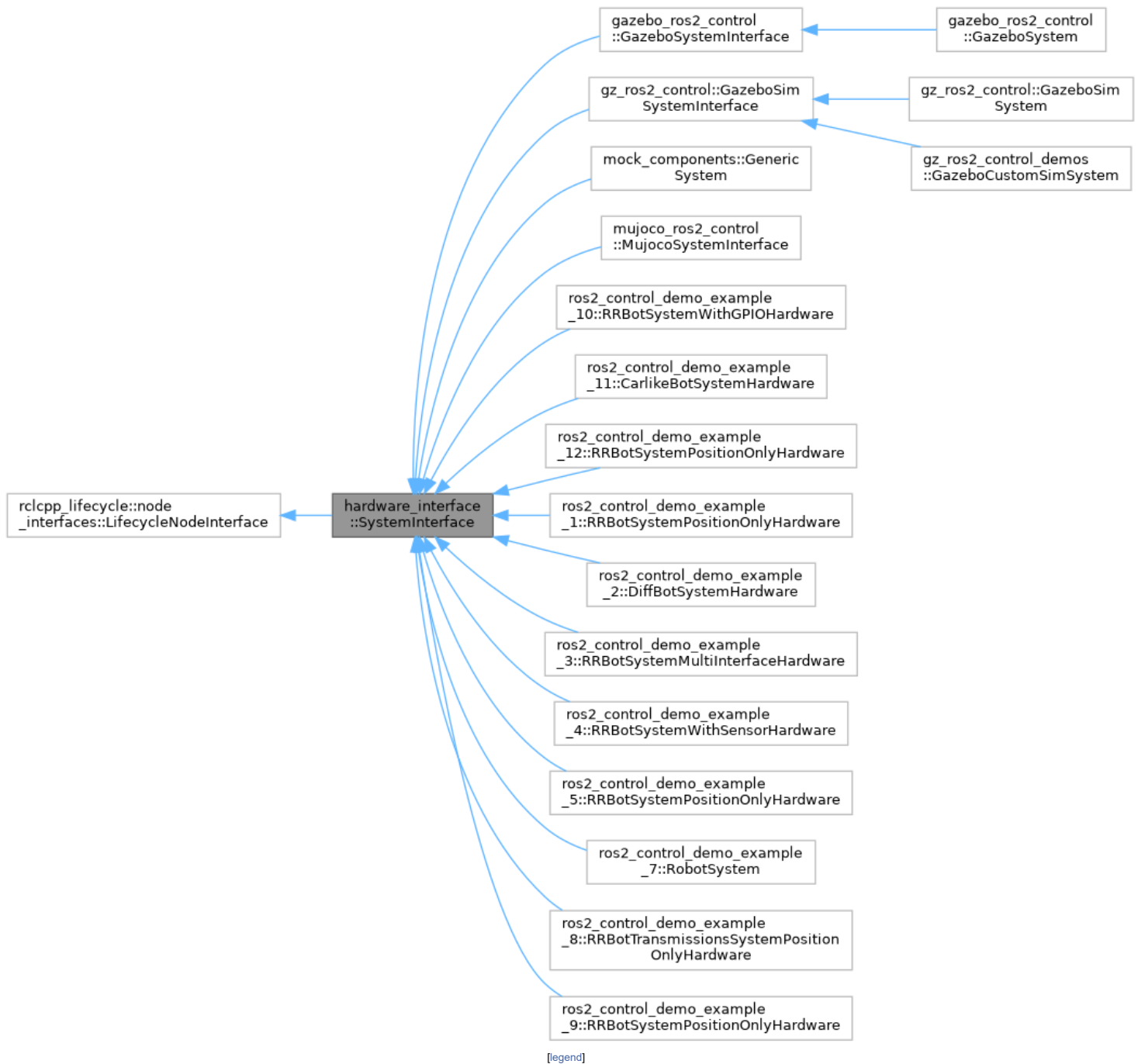


hardware_interface::SystemInterface Class Reference abstract

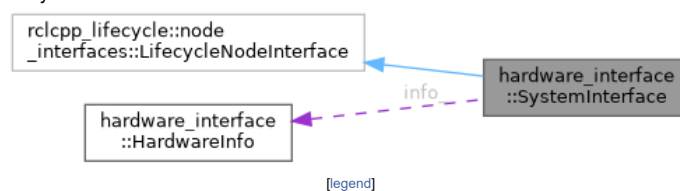
Virtual Class to implement when integrating a complex system into ros2_control. [More...](#)

```
#include <system_interface.hpp>
```

Inheritance diagram for hardware_interface::SystemInterface:



Collaboration diagram for hardware_interface::SystemInterface:



Public Member Functions

SystemInterface (const **SystemInterface** &other)=delete

SystemInterface copy constructor is actively deleted.

	SystemInterface (SystemInterface &&other)=default
virtual CallbackReturn	on_init (const HardwareInfo &hardware_info) Initialization of the hardware interface from data parsed from the robot's URDF.
virtual std::vector< StateInterface >	export_state_interfaces ()=0 Exports all state interfaces for this hardware interface.
virtual std::vector< CommandInterface >	export_command_interfaces ()=0 Exports all command interfaces for this hardware interface.
virtual return_type	prepare_command_mode_switch (const std::vector< std::string > &, const std::vector< std::string > &) Prepare for a new command interface switch.
virtual return_type	perform_command_mode_switch (const std::vector< std::string > &, const std::vector< std::string > &)
virtual return_type	read (const rclcpp::Time &time, const rclcpp::Duration &period)=0 Read the current state values from the actuator.
virtual return_type	write (const rclcpp::Time &time, const rclcpp::Duration &period)=0 Write the current command values to the actuator.
virtual std::string	get_name () const Get name of the actuator hardware.
const rclcpp_lifecycle::State &	get_state () const Get life-cycle state of the actuator hardware.
void	set_state (const rclcpp_lifecycle::State &new_state) Set life-cycle state of the actuator hardware.

Protected Attributes

HardwareInfo	info_
rclcpp_lifecycle::State	lifecycle_state_

Detailed Description

Virtual Class to implement when integrating a complex system into ros2_control.

The common examples for these types of hardware are multi-joint systems with or without sensors such as industrial or humanoid robots.

Methods return values have type rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn with the following meaning:

Returns

- CallbackReturn::SUCCESS method execution was successful.
- CallbackReturn::FAILURE method execution has failed and can be called again.
- CallbackReturn::ERROR critical error has happened that should be managed in "on_error" method.

The hardware ends after each method in a state with the following meaning:

UNCONFIGURED (on_init, on_cleanup): Hardware is initialized but communication is not started and therefore no interface is available.

INACTIVE (on_configure, on_deactivate): Communication with the hardware is started and it is configured. States can be read and command interfaces are available.

As of now, it is left to the hardware component implementation to continue using the command received from the CommandInterfaces or to skip them completely.

FINALIZED (on_shutdown): Hardware interface is ready for unloading/destruction. Allocated memory is cleaned up.

ACTIVE (on_activate): Power circuits of hardware are active and hardware can be moved, e.g., brakes are disabled. Command interfaces available.

Todo:

Implement

- https://github.com/ros-controls/ros2_control/issues/931
- <https://github.com/ros-controls/roadmap/pull/51/files>
- this means in INACTIVE state:
 - States can be read and non-movement hardware interfaces commanded.
 - Hardware interfaces for movement will NOT be available.
 - Those interfaces are: HW_IF_POSITION, HW_IF_VELOCITY, HW_IF_ACCELERATION, and HW_IF_EFFORT.

Constructor & Destructor Documentation

◆ SystemInterface()

hardware_interface::SystemInterface::SystemInterface (const [SystemInterface](#) & other) delete

[SystemInterface](#) copy constructor is actively deleted.

Hardware interfaces are having a unique ownership and thus can't be copied in order to avoid failed or simultaneous access to hardware.

Member Function Documentation

◆ export_command_interfaces()

virtual std::vector< [CommandInterface](#) > hardware_interface::SystemInterface::export_command_interfaces () pure virtual

Exports all command interfaces for this hardware interface.

The command interfaces have to be created and transferred according to the hardware info passed in for the configuration.

Note the ownership over the state interfaces is transferred to the caller.

Returns

vector of command interfaces

Implemented in [gazebo_ros2_control::GazeboSystem](#), [gz_ros2_control::GazeboSimSystem](#), [gz_ros2_control_demos::GazeboCustomSimSystem](#), [mujoco_ros2_control::MujocoSystemInterface](#), [mock_components::GenericSystem](#), [ros2_control_demo_example_1::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_10::RRBotSystemWithGPIOHardware](#), [ros2_control_demo_example_11::CarlikeBotSystemHardware](#), [ros2_control_demo_example_12::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_2::DiffBotSystemHardware](#), [ros2_control_demo_example_3::RRBotSystemMultiInterfaceHardware](#), [ros2_control_demo_example_4::RRBotSystemWithSensorHardware](#), [ros2_control_demo_example_5::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_7::RobotSystem](#), [ros2_control_demo_example_8::RRBotTransmissionsSystemPositionOnlyHardware](#), and [ros2_control_demo_example_9::RRBotSystemPositionOnlyHardware](#).

◆ export_state_interfaces()

virtual std::vector< [StateInterface](#) > hardware_interface::SystemInterface::export_state_interfaces () pure virtual

Exports all state interfaces for this hardware interface.

The state interfaces have to be created and transferred according to the hardware info passed in for the configuration.

Note the ownership over the state interfaces is transferred to the caller.

Returns

vector of state interfaces

Implemented in [gazebo_ros2_control::GazeboSystem](#), [gz_ros2_control::GazeboSimSystem](#), [gz_ros2_control_demos::GazeboCustomSimSystem](#), [mujoco_ros2_control::MujocoSystemInterface](#), [mock_components::GenericSystem](#), [ros2_control_demo_example_1::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_10::RRBotSystemWithGPIOHardware](#), [ros2_control_demo_example_11::CarlikeBotSystemHardware](#), [ros2_control_demo_example_12::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_2::DiffBotSystemHardware](#), [ros2_control_demo_example_3::RRBotSystemMultiInterfaceHardware](#), [ros2_control_demo_example_4::RRBotSystemWithSensorHardware](#), [ros2_control_demo_example_5::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_7::RobotSystem](#), [ros2_control_demo_example_8::RRBotTransmissionsSystemPositionOnlyHardware](#), and [ros2_control_demo_example_9::RRBotSystemPositionOnlyHardware](#).

◆ get_name()

```
virtual std::string hardware_interface::SystemInterface::get_name ( ) const
```

[inline](#) [virtual](#)

Get name of the actuator hardware.

Returns

name.

◆ get_state()

```
const rclcpp_lifecycle::State & hardware_interface::SystemInterface::get_state ( ) const
```

[inline](#)

Get life-cycle state of the actuator hardware.

Returns

state.

◆ on_init()

```
virtual CallbackReturn hardware_interface::SystemInterface::on_init ( const HardwareInfo & hardware_info )
```

[inline](#) [virtual](#)

Initialization of the hardware interface from data parsed from the robot's URDF.

Parameters

[in] [hardware_info](#) structure with data from URDF.

Returns

CallbackReturn::SUCCESS if required data are provided and can be parsed.

CallbackReturn::ERROR if any error happens or data are missing.

Reimplemented in [mujoco_ros2_control::MujocoSystemInterface](#), [mock_components::GenericSystem](#), [ros2_control_demo_example_1::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_10::RRBotSystemWithGPIOHardware](#), [ros2_control_demo_example_11::CarlikeBotSystemHardware](#), [ros2_control_demo_example_12::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_2::DiffBotSystemHardware](#), [ros2_control_demo_example_3::RRBotSystemMultiInterfaceHardware](#), [ros2_control_demo_example_4::RRBotSystemWithSensorHardware](#), [ros2_control_demo_example_5::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_7::RobotSystem](#), [ros2_control_demo_example_8::RRBotTransmissionsSystemPositionOnlyHardware](#), [ros2_control_demo_example_9::RRBotSystemPositionOnlyHardware](#), [gazebo_ros2_control::GazeboSystem](#), [gz_ros2_control::GazeboSimSystem](#), and [gz_ros2_control_demos::GazeboCustomSimSystem](#).

◆ perform_command_mode_switch()

```
virtual return_type hardware_interface::SystemInterface::perform_command_mode_switch ( const std::vector< std::string > & ,
                                                                                     const std::vector< std::string > &
                                                                                     )
```

[inline](#) [virtual](#)

Perform the mode-switching for the new command interface combination.

Note

This is part of the realtime update loop, and should be fast.

All starting and stopping interface keys are passed to all components, so the function should return `return_type::OK` by default when given interface keys not relevant for this component.

Parameters

[in] **start_interfaces** vector of string identifiers for the command interfaces starting.

[in] **stop_interfaces** vector of string identifiers for the command interfaces stopping.

Returns

`return_type::OK` if the new command interface combination can be switched to, or if the interface key is not relevant to this system. Returns `return_type::ERROR` otherwise.

Reimplemented in [gazebo_ros2_control::GazeboSystem](#), [gz_ros2_control::GazeboSimSystem](#), [mujoco_ros2_control::MujocoSystemInterface](#), and [mock_components::GenericSystem](#).

◆ prepare_command_mode_switch()

```
virtual return_type hardware_interface::SystemInterface::prepare_command_mode_switch ( const std::vector< std::string > & ,
                                                                                     const std::vector< std::string > &
                                                                                     )
```

[inline](#) [virtual](#)

Prepare for a new command interface switch.

Prepare for any mode-switching required by the new command interface combination.

Note

This is a non-realtime evaluation of whether a set of command interface claims are possible, and call to start preparing data structures for the upcoming switch that will occur.

All starting and stopping interface keys are passed to all components, so the function should return `return_type::OK` by default when given interface keys not relevant for this component.

Parameters

[in] **start_interfaces** vector of string identifiers for the command interfaces starting.

[in] **stop_interfaces** vector of string identifiers for the command interfaces stopping.

Returns

`return_type::OK` if the new command interface combination can be prepared, or if the interface key is not relevant to this system. Returns `return_type::ERROR` otherwise.

Reimplemented in [mock_components::GenericSystem](#), and [ros2_control_demo_example_3::RRBotSystemMultiInterfaceHardware](#).

◆ read()

```
virtual return_type hardware_interface::SystemInterface::read ( const rclcpp::Time &    time,
                                                             const rclcpp::Duration & period
                                                             )
```

pure virtual

Read the current state values from the actuator.

The data readings from the physical hardware has to be updated and reflected accordingly in the exported state interfaces. That is, the data pointed by the interfaces shall be updated.

Parameters

- [in] **time** The time at the start of this control loop iteration
- [in] **period** The measured time taken by the last control loop iteration

Returns

return_type::OK if the read was successful, return_type::ERROR otherwise.

Implemented in [gazebo_ros2_control::GazeboSystem](#), [gz_ros2_control::GazeboSimSystem](#), [gz_ros2_control_demos::GazeboCustomSimSystem](#), [mujoco_ros2_control::MujocoSystemInterface](#), [mock_components::GenericSystem](#), [ros2_control_demo_example_1::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_10::RRBotSystemWithGPIOHardware](#), [ros2_control_demo_example_11::CarlikeBotSystemHardware](#), [ros2_control_demo_example_12::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_2::DiffBotSystemHardware](#), [ros2_control_demo_example_3::RRBotSystemMultiInterfaceHardware](#), [ros2_control_demo_example_4::RRBotSystemWithSensorHardware](#), [ros2_control_demo_example_5::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_7::RobotSystem](#), [ros2_control_demo_example_8::RRBotTransmissionsSystemPositionOnlyHardware](#), and [ros2_control_demo_example_9::RRBotSystemPositionOnlyHardware](#).

◆ set_state()

```
void hardware_interface::SystemInterface::set_state ( const rclcpp_lifecycle::State & new_state )
```

inline

Set life-cycle state of the actuator hardware.

Returns

state.

◆ write()

```
virtual return_type hardware_interface::SystemInterface::write ( const rclcpp::Time &    time,
                                                                const rclcpp::Duration & period
                                                                )
```

pure virtual

Write the current command values to the actuator.

The physical hardware shall be updated with the latest value from the exported command interfaces.

Parameters

- [in] **time** The time at the start of this control loop iteration
- [in] **period** The measured time taken by the last control loop iteration

Returns

return_type::OK if the read was successful, return_type::ERROR otherwise.

Implemented in [mock_components::GenericSystem](#), [ros2_control_demo_example_7::RobotSystem](#), [gazebo_ros2_control::GazeboSystem](#), [gz_ros2_control::GazeboSimSystem](#), [gz_ros2_control_demos::GazeboCustomSimSystem](#), [mujoco_ros2_control::MujocoSystemInterface](#), [ros2_control_demo_example_1::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_10::RRBotSystemWithGPIOHardware](#), [ros2_control_demo_example_11::CarlikeBotSystemHardware](#), [ros2_control_demo_example_12::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_2::DiffBotSystemHardware](#), [ros2_control_demo_example_3::RRBotSystemMultiInterfaceHardware](#), [ros2_control_demo_example_4::RRBotSystemWithSensorHardware](#), [ros2_control_demo_example_5::RRBotSystemPositionOnlyHardware](#), [ros2_control_demo_example_8::RRBotTransmissionsSystemPositionOnlyHardware](#), and [ros2_control_demo_example_9::RRBotSystemPositionOnlyHardware](#).

The documentation for this class was generated from the following file:

- [ros2_control/hardware_interface/include/hardware_interface/system_interface.hpp](#)